

```

43 if { resp == "200" } {
44     println "tutum hello world is alive and kicking!"
45     docker.withRegistry("${env.REGISTRY}", "docker-hub-entree") {
46         image.push("${git.HASH}")
47         if { "${env.BRANCH_NAME}" == "master" } {
48             image.push("${LATEST}")
49         }
50     }
51     currentBuild.result = "SUCCESS"
52 } else {
53     println "Humans are mortals."
54     currentBuild.result = "FAILURE"
55 }

```

Figure 13: Push the image or Fail the build too are continuation of the block of code in the test stage

```

(Pipeline) withEnv
(Pipeline) {
(Pipeline) withDockerRegistry
13:53:00 Wrote authentication to /var/lib/jenkins/.dockercfg
(Pipeline) {
13:53:00 [tutum-hello-world-anyagent] Running shell script
13:53:01 + docker tag raspamdocker/osfy registry.hub.docker.com/raspamdocker/osfy:be12917
(Pipeline) {
13:53:01 [tutum-hello-world-anyagent] Running shell script
13:53:01 + docker push registry.hub.docker.com/raspamdocker/osfy:be12917
13:53:01 The push refers to a repository (registry.hub.docker.com/raspamdocker/osfy)
13:53:01 5f70bf18a086: Preparing
13:53:01 5f70bf18a086: Preparing
13:53:01 795b6e11d08e: Preparing
13:53:01 3d346273c9f9: Preparing
13:53:01 1a0f9e09313: Preparing
13:53:01 5f70bf18a086: Preparing
13:53:01 a5899b62574: Preparing
13:53:02 5f70bf18a086: Layer already exists
13:53:02 a5899b62574: Layer already exists
13:53:03 3d346273c9f9: Layer already exists
13:53:03 1a0f9e09313: Layer already exists
13:53:03 795b6e11d08e: Layer already exists
13:53:04 be12917: digest:
sha256:75438823963a3f76e0d9f8a42955d1594168f2326d4cb48ba41d4ef7584e785 size: 1773
(Pipeline) // withDockerRegistry
(Pipeline) //
(Pipeline) // withEnv

```

Figure 14: Tag and Push the image that shows the corresponding build console output

```

1 FROM tutum/hello-world
2
3 EXPOSE 80

```

Figure 15: Dockerfile stored at the root of SCM



Figure 16: Stages of the pipeline job

Curl's output is muted (`-s` option) and is redirected to (`-o /dev/null`). For a full explanation of the options, see the man pages of Curl. To keep things brief, we will skip discussing the triple quoted double string – those interested can refer to the Groovy documentation.

Registry upload

Now, it is time to validate the output by running the `curl` command on the port in the host. The expected response is 200, to denote an OK; if not, we will mark the build as *Fail*.

After checking for the response from the `curl` command stored in the variable, `resp`, the pipeline script either pushes the image to the repository hosted in Docker Hub's registry or fails the build (the image is not pushed to the registry).

Lines 45 and 46 tag and push the image to the repository, `raspamdocker/osfy`, hosted in the registry, <https://registry.hub.docker.com>. Recall these were defined in the environment block of the script, as indicated in Figure 2. Credentials to access the registry are stored in Jenkins and accessed using the ID, `docker-hub-entree`.

Also worth mentioning is the check on the branch name, as in Lines 47 to 49, based on which an additional tag is applied to the image and pushed. This approach, i.e., the conditional execution of pipeline code, helps to control the flow of the build.

Assuming that the host port is reachable, the pipeline will tag and push the image, which can be seen in Figure 14.

Authentication to the registry is followed by tagging the image, which is then pushed to the repository.

The final act

Declarative syntax offers a multitude of possibilities as part of the post block – various actions can be grouped and executed if certain conditions are met, i.e., always, failure, changed, and so on (as permitted by the pipeline plugin). These conditions are a reflection of the overall build status as tracked (it can be manually set too) by the pipeline.

In the pipeline script referenced for this article, the build workspace is set to be cleaned always (no matter what the build status is, as influenced by the previous stages).

Successfully running the build job (using the pipeline script) should result in a state of the project's build job, as shown in Figure 16. 

References

- <https://go.cloudbees.com/docs/cloudbees-documentation/cje-user-guide/index.html#docker-workflow>
- <https://www.cloudbees.com/blog/triggering-docker-pipelines-jenkins>
- <https://www.cloudbees.com/blog/orchestrating-workflows-jenkins-and-docker-0>
- <https://github.com/jenkinsci/pipeline-model-definition-plugin/wiki/Controlling-your-build-environment>
- <https://jenkins.io/doc/pipeline/steps/credentials-binding/#withcredentials-bind-credentials-to-variables>
- <https://jenkins.io/doc/book/pipeline/docker/>

By: Ramanathan Muthaiah

The author works for a cyber security firm in Bengaluru. He enjoys the outdoors when he's not churning out Jenkins Pipeline code.